

QuotaLens Setup & Execution Guide

Self-hosted AI Model Quota & Token Usage Dashboard — v1.0.0

1. Project Overview

QuotaLens is a unified management console designed to aggregate, parse, and present rate limit quotas, remaining credits, and historical token usage metrics for developers leveraging multiple AI code assistants. It natively monitors Cursor, Antigravity IDE, GitHub Copilot, Google Gemini Developer API, and OpenRouter across dual Gmail slot mappings.

Service	Address / URL	Credentials
React Dashboard Web Client	http://localhost:5173	None (Direct Access)
FastAPI Swagger API Documentation	http://localhost:8000/docs	None (Direct Access)
sqladmin Interactive DB Panel	http://localhost:8000/admin	Username: admin Password: <i>configured in .env</i>

2. Prerequisites

Before executing QuotaLens, ensure your workstation has the following runtimes installed:

- **Python 3.11 or higher** (for running the FastAPI server and Celery background workers)
- **Node.js 20 or higher** (for running and compiling the Vite React frontend client)
- **Docker Desktop / Compose** (for orchestrating PostgreSQL 16 and Redis 7 databases)

3. Step-by-Step Setup Guide

Step 3.1: Start the Infrastructure Database and Queue Services

Orchestrate the containerized PostgreSQL 16 database and Redis 7 task cache using Docker Compose. A custom port mapping of **5433** is configured for PostgreSQL to prevent collisions with pre-existing local databases.

```
docker compose up -d
```

Step 3.2: Initialize and Setup the Python Virtual Environment

Create a dedicated virtual environment in the backend directory and download dependencies. This ensures packages like SQLAlchemy, Celery, and sqlalchemy are cleanly isolated.

```
cd backend
python -m venv venv
.\venv\Scripts\activate
pip install -r requirements.txt
```

Step 3.3: Configure the Local Environment Variables

Copy the .env.example template file to .env in both the workspace root and backend directory. Edit the variables with your secret key, admin credentials, and API tokens (such as Gemini keys, GitHub tokens, OpenRouter tokens, etc.).

```
copy .env.example .env
copy .env.example backend\.env
```

Step 3.4: Seed the Database with Realistic Demo Logs

Apply initial schema declarations and seed the database tables with 30 days of mock token histories, models limit setups, and test notification parameters.

```
python seed.py
```

Step 3.5: Build and Configure the React Frontend

Navigate to the frontend directory, install npm node packages, and run the client dev server.

```
cd ../frontend
npm install
npm run dev
```

4. Application Execution Scripts

To run the full stack, start these four services simultaneously in separate shell sessions:

1. FastAPI Web Backend (Port 8000):

```
cd backend && .\venv\Scripts\uvicorn main:app --host 127.0.0.1 --port 8000
```

2. Celery Polling Worker:

```
cd backend && .\venv\Scripts\celery -A scheduler.celery_app worker --loglevel=info
```

3. Celery Beat Scheduler:

```
cd backend && .\venv\Scripts\celery -A scheduler.celery_app beat --loglevel=info
```

4. React Client Server (Port 5173):

```
cd frontend && npm run dev
```